

Product Requirements Document

Real-Time Music Interpretation Oscilloscope Visualizer

Working title: **OscilloFlow**

Document status: Product definition / MVP PRD

Primary product type: Local-first, real-time audio-reactive visualizer

Target platforms: Android, iOS, Windows, macOS, with platform-specific capture adapters

Core constraint: OS-agnostic product architecture, not identical audio-capture capability on every OS

1. Product Summary

OscilloFlow is a real-time music interpretation and oscilloscope-art application. The app listens to live audio from an available source, extracts musical and acoustic features, identifies broad instrumental and timbral characteristics, and renders high-fidelity oscilloscope-style visual art that reacts to the music in real time.

The app is not violin-dependent. It is designed for general music, especially instrumental music, and maps musical qualities such as rhythm, energy, bass, transient density, stereo width, harmonic complexity, brightness, and broad instrument family to living geometric visuals.

The product should be local-first. Raw audio should not leave the device by default. The first product should avoid lyric transcription, cloud analysis, social features, account systems, playlist management, and general-purpose AI conversation. The core experience is immediate: open the app, choose an input mode, start listening, and see the music become dynamic oscilloscope art.

2. Product Vision

The product should feel like a modern artistic instrument rather than a generic music visualizer.

The user should be able to play music, open the app, and see the app interpret the music's structure and character as real-time geometry. The visualization should feel physically connected to the sound, not like a pre-rendered animation. The aesthetic should be rooted in oscilloscope logic: XY traces, phase relationships, waveforms, Lissajous figures, glow, persistence, bloom, and signal-driven motion.

The app should answer this promise:

"Turn live music into intelligent oscilloscope-style visual art, locally and in real time."

3. Product Positioning

OscilloFlow sits between three categories:

1. Traditional oscilloscope visualizer
Shows raw signal relationships, waveforms, and Lissajous figures.
2. Music information retrieval tool
Extracts tempo, onsets, spectral qualities, pitch confidence, energy, and broad instrument/timbre features.
3. Artistic live visual instrument
Uses deterministic visual grammar to create expressive, high-fidelity visuals from the interpreted music.

It is not primarily a DJ app, DAW, music player, AI chatbot, music-identification app, lyric analyzer, or generative video model.

4. Problem Statement

Most music visualizers are either too generic, too decorative, too disconnected from the actual audio signal, or too dependent on simple amplitude/frequency effects. They often respond to loudness but do not meaningfully reflect the character of the music.

Users who enjoy music, live visuals, ambient displays, experimental art, electronic music, instrumental music, or oscilloscope aesthetics need a tool that can turn music into visuals that feel physically tied to the sound while still being artistically rich.

The key problem is to create a visualizer that is:

- real-time;
- locally processed;
- musically responsive;
- visually distinctive;
- usable across major platforms;
- honest about OS-specific audio-capture limits;
- not overbuilt into a full music platform.

5. Target Users

Primary Users

Music listeners who want an expressive live visual layer while playing music.

They may use the app while listening through speakers, headphones, desktop audio, phone audio, or imported files. They care about the visual experience more than technical controls.

Secondary Users

Musicians, producers, live performers, and visual artists.

They may use the app for visual inspiration, performance backgrounds, live projection, short clips, or aesthetic experimentation.

Tertiary Users

Technical users interested in oscilloscope art, DSP, audio-reactive rendering, generative visuals, and music information retrieval.

They may want advanced settings, input routing, external microphones, audio interfaces, and export options.

6. Core User Stories

As a listener, I want to open the app while music is playing and see the music transformed into real-time oscilloscope-style art.

As a listener, I want the app to work even when direct system-audio capture is unavailable, by using the microphone or an imported file.

As a user, I want the app to tell me clearly which listening modes are available on my device.

As a user, I want visuals that change depending on whether the music is percussive, string-like, synth-heavy, bass-heavy, piano-like, guitar-like, vocal-heavy, sparse, dense, bright, or dark.

As a privacy-conscious user, I want raw audio to stay on my device by default.

As a visual user, I want high-quality glowing, fluid, responsive visuals without needing to understand DSP.

As an advanced user, I want to tune gain, persistence, bloom, input mode, visual mode, and sensitivity.

As a creator, I want to export short videos or still frames after the real-time experience is stable.

7. Product Goals

Goal 1: Real-Time Visual Response

The app must render audio-reactive visuals with low perceived latency. Visual changes should feel connected to the music's rhythm, energy, transients, and timbre.

Goal 2: Music Interpretation Without Lyrics

The app should detect broad musical qualities and instrument/timbre families without transcribing, interpreting, storing, or analyzing lyrics.

Goal 3: OS-Agnostic Product Model

The product should share one conceptual engine across platforms:

- audio capture adapter;
- audio feature extraction;
- broad classification layer;
- visual grammar engine;
- GPU renderer;
- settings/preset layer.

Platform-specific audio capture should be handled through adapters and presented honestly to users.

Goal 4: Local-First Processing

The MVP should process audio locally. No cloud service should be required for the core experience.

Goal 5: Strong Visual Identity

The app should produce visuals that are recognizably oscilloscope-inspired: XY traces, Lissajous forms, signal trails, glow, phase motion, bloom, and persistence.

8. Non-Goals

The MVP must not include:

- account system;
- cloud sync;
- social feed;
- playlist library;
- music streaming integration as a streaming service;
- DRM bypass;
- lyric transcription;
- lyric interpretation;
- song identification/fingerprinting;
- automatic music downloading;
- DAW editing;
- plugin hosting;
- multi-user collaboration;
- marketplace;
- AI chat companion;
- user prompt-to-video generation;

- full source separation into individual stems;
- exact instrument recognition for every instrument;
- guaranteed capture of audio from every third-party app;
- guarantee that protected or restricted app audio can be captured;
- complex preset marketplace;
- full video editor;
- cloud rendering;
- server-side analytics as a core dependency.

9. Key Product Principle

The app should be honest:

“Available listening modes depend on your device and operating system. Microphone and file modes are broadly available. Capturing audio from other apps is supported only where the operating system and source app allow it.”

This must be reflected in onboarding, permissions, UI copy, help text, and product marketing.

10. Input Modes

The app should support multiple input modes. Not all modes are available on every OS.

10.1 Microphone Mode

The app listens through the device microphone.

This is the most universal live mode. It works with speakers, instruments, ambient music, and external audio sources.

Strengths:

- cross-platform;
- simple user model;
- useful for live music;
- no need to capture another app's audio.

Weaknesses:

- lower fidelity than direct audio capture;
- affected by room acoustics;
- affected by background noise;
- less reliable for headphone listening unless device leakage is audible.

MVP priority: Required.

10.2 File Mode

The user imports an audio file and visualizes it.

Strengths:

- reliable;
- high fidelity;
- useful for testing and export;
- avoids OS capture restrictions.

Weaknesses:

- not the same as spontaneous live listening;
- requires user-owned/imported files.

MVP priority: Required.

10.3 In-App Player Mode

The app plays an imported audio file and visualizes it internally.

Strengths:

- reliable;
- synchronized playback and visualization;
- useful for export;
- avoids system capture restrictions.

Weaknesses:

- does not solve “listen to music from any app” on all platforms.

MVP priority: Required for file playback.

10.4 System / Other-App Audio Capture Mode

The app captures audio being played by other apps or the system where the OS allows it.

Strengths:

- closest to the desired “listen to music, open app, visualize it” experience;
- best user experience where available.

Weaknesses:

- platform-specific;
- permission-heavy;

- source app may block capture;
- not reliable across all apps and OS versions.

MVP priority:

- Android: High priority where OS-supported and allowed.
- Windows: High priority.
- macOS: High priority for desktop version.
- iOS: Not an MVP dependency.

11. Platform Capability Model

The app should not expose the same input promises on every platform. Instead, it should expose a capability model.

Android

Expected input modes:

- microphone;
- file import;
- in-app player;
- other-app playback capture where OS and source app allow it.

Android should be considered the primary mobile platform for “listen to other apps” behavior.

iOS

Expected input modes:

- microphone;
- file import;
- in-app player;
- possible app-internal analysis through AVAudioEngine;
- sound classification through supported native frameworks or custom models.

iOS should not be positioned around unrestricted capture of other apps’ audio in the MVP.

Windows

Expected input modes:

- microphone;
- file import;
- in-app player;
- system output / loopback capture.

Windows should be considered a strong desktop target for direct system-audio visualization.

macOS

Expected input modes:

- microphone;
- file import;
- in-app player;
- system/screen/audio capture where permitted through macOS APIs.

macOS should be considered a strong desktop target but may require more permission and capture configuration than Windows.

Web

Expected input modes:

- microphone;
- file import;
- in-browser audio playback.

Web should be considered a prototype and demo platform, not the strongest route for unrestricted system-audio capture.

12. MVP Scope

The MVP should prove the core experience without becoming a full media platform.

MVP Must Include

1. Microphone input.
2. File import.
3. In-app audio playback for imported files.
4. Real-time oscilloscope-style renderer.
5. Stereo XY mode.
6. Mono phase XY mode.
7. Band-energy XY mode.
8. Temporal persistence and glow.
9. Automatic gain control with clipping protection.
10. Basic musical feature extraction.
11. Broad vocal/instrumental detection.
12. Broad timbre/instrument family classification.
13. Deterministic visual grammar based on audio features.
14. Clear input capability screen.
15. Local-first privacy posture.
16. Basic settings.
17. Preset selection.
18. Still-frame export.

19. Basic performance diagnostics.

MVP Should Include

1. Android playback capture where permitted.
2. Desktop loopback/system audio capture for Windows/macOS if desktop is included in first release.
3. Short video export.
4. Beat-reactive visual modes.
5. Instrument family confidence display in debug mode.
6. Simple onboarding explaining input modes.
7. Low-power mode.

MVP Should Not Include

1. Account creation.
2. Cloud-based analysis.
3. Lyrics.
4. Social sharing backend.
5. Full project library.
6. Real-time prompt-based generation.
7. Complex AI assistant interface.
8. Stem separation.
9. Full DAW-like controls.
10. Marketplace.
11. Music streaming integrations.
12. User profiles.
13. Remote rendering.

13. Functional Requirements

13.1 Start Experience

When the user opens the app for the first time, the app must present a simple input-mode selection:

- Listen through microphone;
- Open audio file;
- Play inside app;
- Listen to other apps, if supported on this platform.

The app must not show unsupported modes as if they are available. Unsupported modes may be shown as disabled with a clear explanation.

Acceptance criteria:

- The user can start a visualization within 30 seconds of first launch using microphone or file mode.
- Unsupported audio-capture modes are not misrepresented.
- Permission prompts are triggered only when the user selects a mode requiring permission.

13.2 Microphone Input

The app must capture live microphone input and route it to the analysis/rendering engine.

Requirements:

- Request microphone permission only when needed.
- Show active input state.
- Show no-signal state when input is silent.
- Apply automatic gain control.
- Warn or visually indicate clipping.
- Continue rendering smoothly during normal input changes.

Acceptance criteria:

- A user can play music through speakers and see visuals react in real time.
- Loud inputs do not permanently distort the visualizer.
- Silence causes the visual to settle rather than freeze or crash.

13.3 File Input

The app must allow the user to import a supported audio file and visualize playback.

Initial supported formats:

- WAV;
- MP3;
- AAC/M4A where platform decoding allows;
- FLAC if feasible without additional complexity.

Acceptance criteria:

- The user can import a local file.
- The app can play, pause, seek, and visualize the file.
- Visualization remains synchronized after seek.

13.4 System / Other-App Audio Capture

Where supported, the app should allow capture of audio being played by other apps or system output.

Requirements:

- Present platform-specific permission flow.
- Explain that capture may not work for protected apps.
- Detect capture failure or silence.
- Offer fallback to microphone or file mode.
- Do not attempt DRM circumvention.

Acceptance criteria:

- On supported platforms, the app can capture allowed playback audio and visualize it.
- If the source app blocks capture, the app explains the failure and suggests another mode.
- The app does not claim universal compatibility.

13.5 Audio Feature Extraction

The app must compute real-time audio features from incoming samples.

Required features:

- RMS amplitude;
- peak amplitude;
- clipping state;
- frequency-band energy;
- spectral centroid / brightness;
- spectral flux;
- transient/onset strength;
- stereo width where stereo is available;
- zero-crossing rate;
- smoothed energy envelope.

Should-have features:

- beat estimate;
- tempo estimate;
- beat phase;
- pitch confidence;
- chroma/key proxy;
- music/speech/voice probability;
- broad instrumental/timbral probabilities.

Acceptance criteria:

- Visuals respond to loudness, bass, treble, transients, and sustained tones.
- Feature extraction does not block rendering.
- The app remains responsive during continuous playback.

13.6 Broad Instrument / Timbre Detection

The app should classify broad musical categories rather than exact instruments.

Initial categories:

- percussion/drums;
- bass;
- strings;

- keys/piano;
- guitar/plucked strings;
- synth/electronic;
- brass/woodwind;
- voice/vocal presence;
- ambient/noise;
- unknown/mixed.

The classifier output should be probabilistic and smoothed. The renderer should not make abrupt visual changes from single-frame classifier changes.

Acceptance criteria:

- The app can distinguish broadly different musical textures well enough to change visual behavior.
- If confidence is low, the app falls back to general audio-reactive visualization.
- Vocal presence can be detected without transcribing lyrics.

13.7 Visual Grammar Engine

The visual grammar engine turns audio features and classifier outputs into rendering parameters.

Required mappings:

- bass energy controls visual scale/depth;
- amplitude controls trace size/intensity;
- transient strength controls pulses/sparks/bloom bursts;
- brightness controls color temperature or edge shimmer;
- stereo width controls horizontal/vertical spread;
- instrumental probabilities blend visual modes;
- silence decays the trace.

Initial visual grammar examples:

- Percussion: radial pulses, sparks, sharp transient geometry.
- Bass: heavy central mass, large-scale movement, slow gravitational expansion.
- Strings: continuous ribbons, curved trails, smooth persistence.
- Piano/keys: crystalline shapes, stepped geometry, note-like impact points.
- Guitar/plucked: harmonic line lattices, short attack trails.
- Synth: smooth Lissajous drift, bloom fields, neon phase shapes.
- Brass/woodwind: strong central curves, breath-like expansion.
- Voice detected: abstract envelope only; no lyric interpretation.

Acceptance criteria:

- Visuals remain coherent when multiple categories are active.
- Category changes feel blended, not abrupt.
- A user can perceive differences between percussive, sustained, bass-heavy, bright, and sparse music.

13.8 Oscilloscope Rendering Modes

The app must include multiple rendering modes.

Required modes: 1. Stereo XY

X axis receives left channel; Y axis receives right channel.

1. Mono Phase XY

X axis receives mono signal; Y axis receives a phase-shifted or delayed version.

2. Band XY

X axis receives one filtered band; Y axis receives another filtered band.

3. Beat-Locked Lissajous

Parametric trace modulated by audio energy and beat phase.

4. Hybrid Instrument Grammar

Visual formulas blend based on classifier outputs.

Acceptance criteria:

- User can switch modes manually.
- App can auto-select a mode based on input type.
- Mono input still produces interesting visuals.

13.9 Visual Quality

The renderer must produce high-fidelity visuals.

Required visual features:

- anti-aliased traces;
- temporal persistence;
- bloom/glow;
- additive light accumulation;
- smooth decay;
- resolution-independent rendering;
- stable frame pacing.

Should-have visual features:

- multiple color palettes;
- intensity-based trace width;
- short and long trail modes;
- background fade control;
- low-power rendering mode.

Acceptance criteria:

- Visuals run smoothly at target frame rate on supported devices.
- Visuals do not strobe excessively by default.
- The app remains usable on both small phones and desktop displays.

13.10 Presets

The app should include a small set of curated presets.

Initial presets:

- Classic Oscilloscope;
- Fire Trace;
- Neon Lissajous;
- Deep Bass Field;
- String Ribbons;
- Percussion Sparks;
- Synth Grid;
- Minimal Monochrome.

Acceptance criteria:

- Presets change visual behavior meaningfully.
- Presets are not just color changes.
- User can reset to default.

13.11 Settings

Initial settings:

- input gain;
- auto-gain on/off;
- sensitivity;
- persistence length;
- bloom strength;
- visual mode;
- color palette;
- low-power mode;
- show debug analysis on/off;
- export quality.

Acceptance criteria:

- Settings are understandable to non-technical users.
- Advanced controls are not required for the first successful experience.
- Settings persist locally.

13.12 Export

MVP export:

- still image capture.

Post-MVP export:

- short video clip export;
- transparent-background export if feasible;
- preset export/import.

Acceptance criteria:

- User can save a still image without interrupting the live session.
- Export does not require account creation.
- Exported image reflects the current visual state.

14. Non-Functional Requirements

14.1 Performance

Target performance:

- 60 fps visual rendering on supported devices.
- Audio analysis latency low enough to feel connected to playback.
- No audio callback blocking.
- No avoidable memory allocation inside real-time audio callback.
- Graceful degradation on weaker devices.

Performance priority: 1. Stable audio capture. 2. Stable rendering. 3. Reliable feature extraction. 4. Classification. 5. Export.

If performance is constrained, classification and bloom should degrade before audio capture or base visualization.

14.2 Latency

The app should feel real-time. The user should perceive visual changes as linked to music.

Targets:

- Microphone/file visual response: preferably under 100 ms perceived latency.
- Classifier updates: acceptable at slower intervals, such as 250 ms to 2 seconds.
- Visual trace response: fast.
- Instrument grammar transitions: smoothed.

14.3 Reliability

The app should not fail silently.

Required failure states:

- no input;
- permission denied;
- protected audio source;
- unsupported mode;
- file decode failure;
- audio device unavailable;
- GPU/rendering fallback;
- classifier unavailable.

Each failure state should offer a practical fallback.

14.4 Privacy

The default privacy model must be local-first.

Requirements:

- No raw audio upload by default.
- No lyric transcription.
- No background listening without explicit active session.
- No microphone activation before permission and user action.
- Clear active listening indicator.
- Local settings storage.
- Clear explanation of what is processed.

Optional future cloud features must be opt-in and outside MVP scope.

14.5 Accessibility

Requirements:

- Avoid mandatory flashing or strobing modes.
- Provide reduced-motion mode.
- Provide brightness control.
- Provide color palettes that remain useful for color-vision differences.
- Make controls usable without needing to understand audio engineering terms.
- Provide clear stop/listening state.

14.6 Battery and Thermal Management

Mobile requirements:

- Low-power mode.
- Lower frame-rate option.
- Bloom reduction option.
- Classifier throttling.
- Stop processing when inactive unless explicitly allowed.
- Warn or degrade gracefully under thermal pressure where platform signals exist.

15. Architecture Requirements

The app should be built as a shared conceptual pipeline with platform-specific adapters.

15.1 Core Pipeline

```
Input Adapter
→ Audio Buffer / Ring Buffer
→ Preprocessing
→ Feature Extraction
→ Classification Layer
→ Visual Grammar Engine
→ Renderer
→ Export / Capture
```

15.2 Input Adapter Layer

Responsibilities:

- microphone capture;
- file decode;
- in-app playback;
- platform-specific system audio capture where available;
- permission handling;
- device selection;
- sample-rate handling;
- stereo/mono detection.

15.3 Audio Preprocessing Layer

Responsibilities:

- resampling where needed;
- channel conversion;
- DC offset removal;

- normalization;
- automatic gain control;
- limiter;
- bandpass/filterbank preparation;
- ring buffer writes.

15.4 Feature Extraction Layer

Responsibilities:

- compute fast frame-level features;
- compute slower musical features;
- smooth noisy values;
- expose normalized feature values to renderer;
- avoid blocking audio capture.

15.5 Classification Layer

Responsibilities:

- broad instrument/timbre prediction;
- vocal presence detection;
- confidence scoring;
- smoothing and hysteresis;
- fallback to unknown/mixed state.

The classifier should not be allowed to directly draw visuals. It only provides structured signals.

15.6 Visual Grammar Layer

Responsibilities:

- map features and classification outputs to visual parameters;
- blend visual modes;
- handle preset logic;
- keep rendering deterministic;
- avoid abrupt changes.

15.7 Renderer

Responsibilities:

- render XY traces;
- render additive glow;
- render persistence;
- handle bloom;
- handle resizing;
- maintain stable frame pacing;

- support low-power mode.

15.8 Local Storage

Local storage should hold:

- settings;
- presets;
- recent files if user permits;
- export history if implemented.

No account system is required.

16. Recommended Technical Approach

16.1 MVP Development Approach

The safest path is:

1. Build an offline/file-based prototype first.
2. Add microphone live mode.
3. Add high-quality renderer.
4. Add feature extraction.
5. Add broad classifier.
6. Add platform-specific system capture.
7. Add export.

This order avoids building platform capture complexity before proving the core visual engine.

16.2 Shared Core Recommendation

Use a shared audio/analysis/rendering design, but do not force one UI framework to solve all platform problems.

Reasonable core options:

- C++ core for maximum portability and easier integration with native mobile/desktop audio APIs.
- Rust core for safety and modern cross-platform rendering, especially on desktop.
- TypeScript/Web Audio/WebGL prototype for fastest visual iteration.

For production, platform-specific native adapters will still be needed.

16.3 Platform Implementation Recommendation

Android:

- Kotlin for app/UI.
- Native audio/capture adapter where needed.

- TensorFlow Lite or native model runtime for classification.
- OpenGL ES or Vulkan renderer.
- Android playback capture where available.

iOS:

- Swift/SwiftUI.
- AVAudioEngine for microphone/in-app/file audio.
- Core ML/Sound Analysis for classification.
- Metal renderer.
- Do not depend on unrestricted other-app audio capture.

Windows:

- Native desktop shell or cross-platform desktop shell.
- WASAPI loopback.
- GPU renderer via wgpu, DirectX, OpenGL, or Vulkan abstraction.

macOS:

- Swift/native or cross-platform shell.
- Microphone/file input.
- Screen/audio capture where permitted.
- Metal or cross-platform GPU layer.

Web prototype:

- TypeScript.
- Web Audio API.
- AudioWorklet.
- WebGL/WebGPU.
- File and microphone modes.

17. AI / ML Scope

The MVP should use audio ML/classification, not a general conversational AI assistant.

In Scope

- broad instrument/timbre classification;
- vocal presence detection;
- mood/energy descriptors if locally feasible;
- structured visual scene selection;
- local inference where possible.

Out of Scope

- lyric transcription;

- lyric meaning analysis;
- open-ended chat;
- cloud LLM dependency;
- prompt-generated visuals;
- exact instrument separation;
- automatic song identification;
- storing audio for later AI analysis.

Optional Later Layer

A local assistant-style layer may be added later, but it must consume structured audio-analysis data, not raw open-ended audio, and it must output constrained visual-state parameters.

Example input:

```
{
  "tempo": 126,
  "energy": 0.82,
  "brightness": 0.74,
  "dominant_categories": ["synth", "drums", "bass"],
  "voice_present": false
}
```

Example output:

```
{
  "visual_scene": "neon_lissajous_grid",
  "motion": "beat_locked",
  "geometry": "radial_pulse",
  "confidence": 0.77
}
```

The renderer should remain deterministic and safe even if this layer is disabled.

18. Visual Mapping Specification

18.1 Core XY Mapping

Stereo XY:

```
x = left_channel_sample
y = right_channel_sample
```

Mono Phase XY:

```
x = mono_sample  
y = phase_shifted_or_delayed_mono_sample
```

Band XY:

```
x = low_mid_band_signal  
y = high_mid_band_signal
```

Beat-Locked Parametric XY:

```
x = sin(phase_a) × amplitude × modulation  
y = sin(phase_b) × amplitude × modulation
```

18.2 Feature-to-Visual Mapping

Amplitude:

- trace size;
- brightness;
- line thickness.

Bass energy:

- scale;
- central mass;
- depth;
- slow expansion.

Transient strength:

- sparks;
- pulses;
- shockwaves;
- bloom bursts.

Spectral brightness:

- color temperature;
- edge shimmer;
- fine particles.

Stereo width:

- horizontal/vertical spread;
- spatial openness.

Beat phase:

- rhythmic expansion/contraction;
- grid pulses;
- radial timing.

Vocal presence:

- lyric-free envelope abstraction;
- possible visual softening or masking;
- no transcription.

18.3 Instrument/Timbre Mapping

Percussion:

- short-lived, angular, high-energy marks.

Bass:

- heavy, low-frequency, large-scale movement.

Strings:

- continuous flowing curves and ribbon-like trails.

Piano/keys:

- crystalline, stepped, geometric impacts.

Guitar/plucked strings:

- harmonic lattices and short attack trails.

Synth/electronic:

- smooth Lissajous drift, neon glow, parametric forms.

Brass/woodwind:

- breath-driven expansion, central arcs, bold curves.

Voice:

- detected as presence only; no words, no semantic interpretation.

Unknown/mixed:

- default oscilloscope mode driven by raw features.

19. UX Requirements

19.1 Main Screen

The main screen should prioritize the visual.

Required elements:

- active visual canvas;
- current input mode;
- start/stop control;
- preset selector;
- minimal settings access;
- input status indicator.

The UI should not clutter the visual with excessive technical readouts.

19.2 Input Selection Screen

The input screen should show:

- Listen through microphone;
- Open file;
- Play inside app;
- Listen to other apps/system audio where supported.

Each mode should include:

- availability;
- permission requirement;
- quality expectation;
- known limitations.

19.3 Debug / Technical Overlay

Optional debug overlay:

- RMS;
- clipping;
- tempo estimate;
- dominant categories;
- vocal presence;
- frame rate;
- input latency estimate.

This should be off by default.

19.4 User Copy Principles

Use clear, honest language.

Good: "Some apps block audio capture. Try microphone or file mode instead."

Bad: "This app visualizes any music from any app."

Good: "Voice detected. Lyrics are not analyzed."

Bad: "AI understands the song."

20. Permissions

The app should request only permissions needed for the selected mode.

Potential permissions:

- microphone;
- file access;
- screen/audio capture permissions where applicable;
- media projection permission on Android;
- save/export permission where applicable.

Permission explanations must be specific and plain-language.

21. Data Handling

21.1 Audio

Raw audio:

- processed locally;
- not uploaded by default;
- not stored unless user explicitly records/exports;
- not used for lyric transcription.

21.2 Analysis Data

Analysis data may include:

- tempo estimate;
- energy values;
- classifier probabilities;
- visual settings.

This data should remain local unless the user exports or shares it.

21.3 Settings

Settings are stored locally.

21.4 Analytics

No analytics are required for MVP.

If analytics are later added, they should not collect raw audio or inferred song content by default.

22. Success Metrics

MVP Functional Success

- User can start microphone visualization quickly.
- User can import and visualize a file.
- Visuals run smoothly on target devices.
- Visuals respond clearly to rhythm, amplitude, bass, treble, and transients.
- App detects vocal presence without transcription.
- App uses fallback modes when system capture is unavailable.

Product Experience Success

- Users understand which input mode to use.
- Users do not feel misled by platform limitations.
- Visual output feels meaningfully connected to the music.
- Presets produce distinct visual behavior.
- The app feels useful without advanced settings.

Technical Success

- Audio callback remains stable.
- Rendering frame rate remains stable.
- Classifier can be disabled without breaking visuals.
- Unsupported capture modes fail gracefully.
- No raw audio leaves device by default.

23. Acceptance Criteria for MVP Release

The MVP is acceptable when all of the following are true:

1. Microphone mode works on all target MVP platforms.
2. File import and in-app playback work reliably.
3. The renderer produces stable oscilloscope-style visuals.
4. At least three XY/rendering modes are implemented.
5. At least five presets are implemented.
6. Basic feature extraction drives visible changes.

7. Broad vocal/instrumental detection works well enough to avoid lyric interpretation.
8. Broad timbre/instrument categories influence visual grammar.
9. Unsupported capture modes are clearly communicated.
10. System/other-app capture is implemented only where technically and legally permitted.
11. App does not require an account.
12. App does not require cloud processing.
13. App does not upload raw audio by default.
14. App has a clear stop/listening state.
15. The app handles silence, clipping, permission denial, and capture failure gracefully.
16. Still-image export works.
17. Performance is acceptable on target test devices.

24. Release Phases

Phase 0: Technical Spike

Goal: Prove that the core visual effect works.

Deliverables:

- offline file visualizer;
- stereo XY mode;
- mono phase mode;
- glow/persistence renderer;
- simple gain control.

Do not include:

- AI;
- system capture;
- export beyond screenshots;
- mobile app polish.

Phase 1: MVP Core

Goal: Create usable local visualizer.

Deliverables:

- microphone input;
- file input;
- in-app playback;
- feature extraction;
- multiple visual modes;
- presets;
- basic settings;
- still export;

- privacy copy.

Phase 2: Interpretation Layer

Goal: Make the app instrument/timbre-aware.

Deliverables:

- broad classifier;
- vocal presence detection;
- visual grammar blending;
- debug overlay;
- classifier confidence handling.

Phase 3: Platform Capture Expansion

Goal: Add “listen to other apps/system audio” where supported.

Deliverables:

- Android playback capture;
- Windows loopback capture;
- macOS capture route;
- clear fallbacks;
- capture failure handling.

iOS should remain microphone/file/in-app-player unless a reliable approved capture path is confirmed.

Phase 4: Creator Features

Goal: Support sharing and performance use.

Deliverables:

- short video export;
- better preset management;
- performance mode;
- external display support where relevant.

Do not add social feed or accounts unless a later product strategy requires it.

25. Risks and Mitigations

Risk 1: User expects all apps on all OSes to be capturable

Mitigation: Use capability-based input selection. Never promise universal other-app audio capture.

Risk 2: Visuals feel generic

Mitigation: Keep oscilloscope XY logic central. Use classification to modulate visual grammar, not replace signal-driven rendering.

Risk 3: AI layer causes latency or instability

Mitigation: Make the renderer work without AI. Run classification slower than rendering. Smooth classifier outputs.

Risk 4: Classifier accuracy is imperfect

Mitigation: Use broad categories, confidence thresholds, and fallback visuals. Avoid exact instrument claims.

Risk 5: Mobile battery drain

Mitigation: Low-power mode, classifier throttling, reduced bloom, lower frame rate, pause when inactive.

Risk 6: Privacy concerns

Mitigation: Local-first by default, no lyric transcription, no raw audio upload, clear active listening state.

Risk 7: Protected audio cannot be captured

Mitigation: Explain clearly and offer microphone/file modes.

Risk 8: Overbuilding into a music platform

Mitigation: No accounts, no playlists, no cloud library, no social feed, no streaming integration in MVP.

26. Open Questions

1. Which platform should be first: Android, desktop, or web prototype?
2. Is the first public product mobile-first or desktop-first?
3. Should the app support only instrumental-first visualization, or general music with vocal detection?
4. Is short video export required for MVP or Phase 4?
5. Should the product use a stylized brand/aesthetic from day one, or start with neutral technical visuals?
6. What minimum device class should be supported?
7. Should system capture be a core promise or an advanced mode?

27. Recommended MVP Definition

The most realistic MVP is:

- local-first;
- no account;
- no cloud;
- microphone mode;
- file mode;
- in-app playback;
- real-time oscilloscope renderer;
- broad audio feature extraction;
- broad vocal/instrumental/timbre detection;
- deterministic visual grammar;
- still export;
- Android/desktop system capture only where supported and after the core engine works.

The product should not be sold as “AI understands any song.”

It should be sold as:

“Real-time oscilloscope art that listens to music, detects its musical character, and turns it into living geometry on your device.”

28. Out-of-Scope Until Proven Necessary

The following should remain explicitly out of scope unless user demand or technical validation justifies them:

- cloud AI;
- lyric interpretation;
- song recognition;
- direct streaming service integration;
- social sharing backend;
- user accounts;
- preset marketplace;
- full editor timeline;
- source separation;
- exact instrument ID;
- multi-device sync;
- collaborative live rooms;
- paid creator economy;
- general AI assistant persona.

29. Final Product Boundary

OscilloFlow is not a replacement for a music player, DAW, AI assistant, or video generator.

It is a focused local-first visual instrument:

1. Capture or receive audio.
2. Analyze the music locally.
3. Interpret broad musical character.
4. Render high-fidelity oscilloscope-inspired visuals.
5. Respect platform limitations.
6. Avoid lyrics and cloud dependency.
7. Stay simple enough to build and validate.